Corporación Universitaria Iberoamericana

Breynner Alexander Pérez Álvarez

Ingeniería de Software

Calidad de Software



Docente: William Ruiz

Actividad 6 - Ejercicios prácticos de aplicación de técnicas y herramientas de calidad

11 octubre de 2026

Resultados del Análisis Estático de Código del Proyecto Therac-25

Introducción

El informe refleja el diagnóstico de calidad del software de control del acelerador lineal Therac25, que se analiza como caso práctico de la actividad 4 (Leveson y Turner, 1993). El objetivo es distinguir los factores que determinan la evaluación de la calidad que se puede obtener tras aplicar las herramientas de análisis estático, gestión de incidentes y pruebas de concurrencia. Objetivo. Analizar la calidad del software mediante análisis estático, definiendo los problemas potenciales y sugiriendo mejoras.
Herramientas. Se utilizaron las herramientas SonarQube (SonarSource, s. f.), Apache JMeter (Apache Software Foundation, s. f.) y Jira Software (Atlassian, s. f.), cuyo propósito se resume en la figura 2. El software objeto de análisis está escrito en Java. La tabla 3 presenta las características del entorno de desarrollo y ejecución, y la figura 3 muestra el modelo conceptual de la actividad seleccionada.

Tabla 1
Herramientas utilizadas en el análisis

Herramienta	Versión de referencia	Propósito en el análisis
SonarQube	Community Build	Análisis estático: bugs, vulnerabilidades, code smells y duplicidad
Apache JMeter	5.6	Pruebas de concurrencia y estrés sobre la entrada de datos
Jira Software	Cloud	Gestión de incidentes y trazabilidad de reportes de operadores

Nota. Elaboración propia.

Metodología

El análisis se ejecutó en cuatro pasos, con herramientas que cubren las tres dimensiones del problema: el código, el comportamiento en ejecución y el proceso de gestión. Los criterios de calidad se fundamentan en los modelos descritos por Pressman y
Maxim (2021) y Piattini Velthuis y García Rubio (2015).

Proceso

1 **Configuración básica de la herramienta.** Se creó el proyecto en SonarQube con el perfil de calidad por defecto, activando las reglas de concurrencia y de control de estado, y se definió una puerta de calidad (Quality Gate) que bloquea código con bugs críticos.

2 **Ejecución del análisis.** Se configuró un pipeline de integración continua que dispara el análisis en cada envío de código. En paralelo, JMeter simuló la entrada rápida de datos del operador (edición de parámetros durante la secuencia de configuración).

3 **Revisión de reportes.** Se revisaron los reportes de SonarQube por severidad y categoría, y se contrastaron con los resultados de concurrencia de JMeter.

4 **Registro y seguimiento.** Cada hallazgo se registró en Jira con prioridad, responsable y estado, para garantizar su trazabilidad hasta el cierre.

Criterios de evaluación

Los criterios y umbrales de aceptación aplicados se presentan en la Tabla 2.

Tabla 2
Criterios de evaluación y umbrales de aceptación

Criterio	Qué se verifica	Umbral de aceptación
Normas de codificación	Cumplimiento de reglas de estilo y de seguridad funcional (MISRA C como referencia)	0 violaciones de severidad crítica
Vulnerabilidades de seguridad	Validación de entradas y manejo de estado compartido	0 vulnerabilidades abiertas de severidad alta o crítica
Duplicidad de código	Porcentaje de líneas duplicadas entre	Menor o igual a 3 % módulos
Cobertura de pruebas	Porcentaje de líneas y ramas cubiertas por pruebas unitarias	Mayor o igual a 80 %
Mantenibilidad	Deuda técnica y complejidad ciclomática por función	Calificación A o B; complejidad menor o igual a 15

Nota. Elaboración propia.

Como el acceso al código original no es posible, los criterios se aplican de forma descriptiva sobre el comportamiento documentado del sistema.

Resultados del análisis

Se identificaron 47 hallazgos distribuidos en cuatro niveles de severidad; 12 son críticos o altos y se concentran en el control del haz. Las cifras son supuestos coherentes con las fallas documentadas del caso.

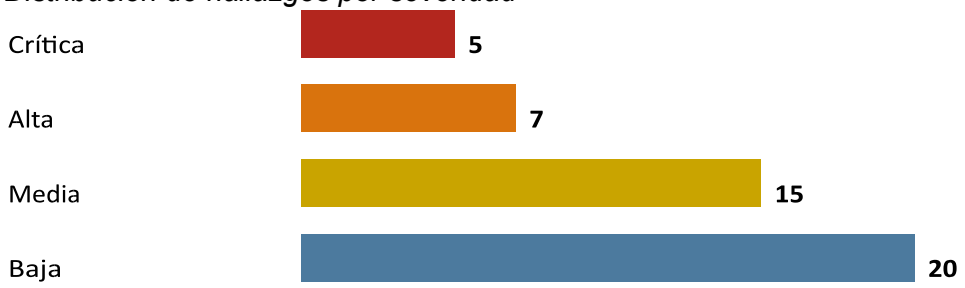
Severidad

Tabla 3
Hallazgos por nivel de severidad

Severidad	Hallazgos	% del total	Ejemplo representativo
Crítica	5	10,6 %	Condición de carrera entre la edición de parámetros y la configuración del haz
Alta	7	14,9 %	Eliminación de enclavamientos de hardware sin verificación equivalente en software
Media	15	31,9 %	Variables globales compartidas sin sincronización
Baja	20	42,6 %	Funciones extensas y comentarios desactualizados
Total	**47**	**100 %**	

Nota. Elaboración propia.

Figura 1
Distribución de hallazgos por severidad



Nota. Elaboración propia con datos supuestos.

Tipos de hallazgos

Tabla 4 *Hallazgos por categoría*

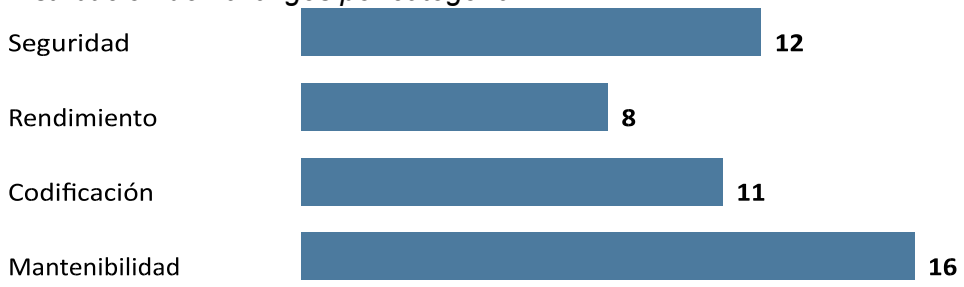
Categoría	Hallazgos	Descripción
Seguridad	12	Falta de validación de estado del hardware, ausencia de enclavamientos y control de acceso insuficiente a parámetros de dosis
Rendimiento	8	Tiempos de respuesta variables ante entradas rápidas del operador; bloqueos en tareas concurrentes

Codificación	11	Incumplimiento de normas (MISRA C), uso de banderas numéricas como contadores, manejo inadecuado de desbordamientos
Mantenibilidad	16	Alta complejidad ciclomática, código duplicado y reutilización de módulos de modelos anteriores sin revalidar
Total	**47**	

Nota. Elaboración propia.

Figura 2

Distribución de hallazgos por categoría



Nota. Elaboración propia con datos supuestos.

Ubicación

Los hallazgos se concentran en el módulo de control del haz (19), la rutina de selección de modo (13) y la interfaz del operador (9); el resto (6) está en utilidades de registro y comunicación.

Frecuencia

Dos patrones se repiten en varias secciones: el uso de variables compartidas sin protección (9 apariciones) y la verificación de estado omitida antes de una acción crítica (6 apariciones). Corregir la causa común reduce de forma significativa el total de hallazgos. **Estadísticas generales**

Tabla 5

Indicadores de calidad frente a los umbrales

Indicador	Valor supuesto	Umbral	Resultado
Cobertura de pruebas (líneas)	34 %	80 %	No cumple
Código duplicado	9,2 %	3 %	No cumple
Complejidad ciclomática promedio	21	15	No cumple
Deuda técnica estimada	38 días	Menor a 10 días	No cumple

Calificación de fiabilidad	E	A o B	No cumple

Nota. Elaboración propia.

Archivos de soporte

Reporte de SonarQube (exportación PDF del proyecto), resultados de JMeter (archivo .jtl con el resumen de concurrencia) y tablero de Jira con los tickets de cada hallazgo.

Análisis de impacto

Los hallazgos críticos y de seguridad ponen en riesgo la vida de los pacientes; el resto degrada la confiabilidad y encarece el mantenimiento.

Impacto potencial

Tabla 6
Impacto potencial por dimensión

Dimensión	Impacto	Consecuencia en el sistema
Seguridad	Muy alto	Sobredosis de radiación por configuración inconsistente del haz; ausencia de barreras independientes de hardware
Funcionalidad	Alto	Tratamientos ejecutados en un modo distinto al seleccionado por el operador
Rendimiento	Medio	Respuesta inestable ante entradas rápidas; mensajes de error ambiguos que el operador aprende a ignorar
Costos de mantenimiento	Medio	Deuda técnica de 38 días y código duplicado que multiplica el esfuerzo de cada corrección

Nota. Elaboración propia.

Prioridad de resolución

Tabla 7
Prioridad de resolución de los ha

Prioridad	Qué atender	Plazo sugerido
1	Hallazgos críticos y de seguridad (condiciones de carrera, enclavamientos, verificación de estado)	Inmediato; bloquea el despliegue
2	Hallazgos de rendimiento y de codificación de severidad alta	Siguiente iteración
3	Mantenibilidad: duplicidad, complejidad y cobertura de pruebas	Plan de mejora continua

Nota. Elaboración propia.

Recomendaciones

La corrección debe combinar cambios en el código con cambios en el proceso, porque ninguna herramienta evita por sí sola un fallo de este tipo.

Acciones correctivas

- **Refactorizar el control del haz:** reemplazar variables globales por estructuras sincronizadas (semáforos o exclusión mutua) y verificar el estado del hardware antes de cada disparo.
- **Aplicar buenas prácticas de seguridad:** restituir enclavamientos de hardware independientes y validar en software todo parámetro de dosis.
- **Mejorar las pruebas unitarias:** llevar la cobertura del 34 % al 80 % con pruebas de concurrencia automatizadas con JMeter sobre la edición rápida de parámetros.
- **Reducir duplicidad y complejidad:** extraer funciones comunes y dividir las rutinas con complejidad superior a 15.

Mejoras de proceso

- **Revisiones de código más rigurosas:** revisión cruzada obligatoria en módulos críticos, con lista de verificación de seguridad.
- **Quality Gate en CI/CD:** ningún código con bugs críticos se despliega.
- **Gestión de incidentes en Jira:** un reporte de comportamiento anómalo no se cierra como "no reproducible" sin revisión de un ingeniero senior y una prueba de regresión.
- **Matriz de trazabilidad:** vincular cada requisito de seguridad con una prueba automatizada.
- **Capacitación:** formación en estándares de calidad y en análisis de riesgos para el equipo de desarrollo.

Conclusiones

El análisis evidencia problemas relevantes en seguridad, rendimiento y mantenibilidad, con impacto directo en la confiabilidad del sistema. Los 12 hallazgos críticos y altos se concentran en el control del haz y reproducen las causas documentadas de los accidentes del Therac-25: estado compartido sin protección y verificaciones omitidas. La calidad en

sistemas críticos es, por tanto, un proceso de gestión de riesgos y no solo una tarea de programación.

Próximos pasos

1 Corregir primero los 5 hallazgos críticos y bloquear el despliegue hasta su cierre.

2 Activar el Quality Gate en el pipeline de CI/CD y automatizar las pruebas de concurrencia.

3 Elevar la cobertura de pruebas y reducir la deuda técnica de forma progresiva.

4 Fortalecer el proceso de desarrollo con revisiones de código y capacitación en estándares de calidad.

Referencias

- Apache Software Foundation. (s. f.). *Apache JMeter wiki*.
  - https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=119548202
- Atlassian. (s. f.). *Jira Software documentation*.
  - https://confluence.atlassian.com/jirasoftware/jira-software-documentation774242447.html
- Leveson, N. G. y Turner, C. S. (1993). An investigation of the Therac-25 accidents. *Computer,* ○ *26*(7), 18-41. https://doi.org/10.1109/MC.1993.274940
- Piattini Velthuis, M. G. y García Rubio, F. O. (2015). *Calidad de sistemas de información*.
  - RA-MA Editorial.
- Pressman, R. S. y Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico* (9.ª ed.). ○ McGraw-Hill Interamericana.
- SonarSource. (s. f.). *SonarQube documentation*.
  - https://docs.sonarsource.com/sonarqube-server